

Basic (your first FastAPI route) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Every version below is a complete `main.py` you can run with `uvicorn main:app --reload` and test with `curl localhost:8000/health`. Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/health")
def health() -> dict:
    return {"status": "ok"}
```

Expected output (`curl localhost:8000/health`):

```
{"status": "ok"}
```

This works, is correct, and is genuinely how many real `/health` routes start out. It's missing a typed response model and doesn't check anything beyond "the process can respond" — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a typed response with `response_model`

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class HealthResponse(BaseModel):
    status: str

@app.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    """Liveness check: is the process up and able to answer HTTP requests at all.
    return HealthResponse(status="ok")
```

Expected output:

```
{"status": "ok"}
```

The JSON reply looks identical to Basic's, but `http://localhost:8000/docs` now shows a documented `HealthResponse` schema for this route, and FastAPI would reject (with a server-side error, loudly, not silently) any code path that tried to return something that doesn't match `HealthResponse` — a bug it would otherwise never catch.

Approach 2 — a test file using `TestClient`, instead of `curl`

```
# test_main.py
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_health_returns_ok():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}
```

Expected output (`pytest test_main.py`):

```
1 passed
```

`TestClient` calls your routes directly in-process — no server, no port, no network — which is why this test runs in milliseconds and works in CI with no setup.

Difference from Basic: Approach 1 adds a real, checked response shape (`HealthResponse`) instead of an unchecked dict — the exact same upgrade `env_parsing` made from a bare dict return to a typed one, just at the HTTP boundary instead of a function's return value. Approach 2 doesn't change `main.py` at all; it shows the other half of doing this properly — a route that's actually tested, not just manually curled once and forgotten.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — separate liveness and readiness routes

```
from fastapi import FastAPI, Response
from pydantic import BaseModel

app = FastAPI()

class HealthResponse(BaseModel):
    status: str

# In a real app this would be set by real startup code, or checked live
# against an actual database connection. Faked here to keep the exercise small.
database_is_reachable = True

@app.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    """Liveness: is the process itself up. Never checks a dependency."""
    return HealthResponse(status="ok")

@app.get("/ready", response_model=HealthResponse)
def ready(response: Response) -> HealthResponse:
    """Readiness: can this service actually do its job right now."""
    if not database_is_reachable:
        response.status_code = 503
        return HealthResponse(status="not ready")
    return HealthResponse(status="ok")
```

Expected output with `database_is_reachable = True` :

```
GET /health -> 200 {"status":"ok"}
GET /ready -> 200 {"status":"ok"}
```

Expected output after setting `database_is_reachable = False` by hand:

```
GET /health -> 200 {"status":"ok"}
GET /ready -> 503 {"status":"not ready"}
```

`/health` staying at 200 while `/ready` drops to 503 is the entire point: a restart won't fix a dead database, so a system watching these two routes should stop sending this instance new traffic (via `/ready`) without killing and restarting a perfectly healthy process (which `/health` still reports as fine).

Approach 2 — `APIRouter`, plus tests for both routes

Once you have more than one or two routes, grouping them with `APIRouter` instead of decorating straight onto `app` keeps `main.py` small — this is the pattern `chat_route` and the Build Task both lean on.

```
# routes.py
from fastapi import APIRouter, Response
from pydantic import BaseModel

router = APIRouter()

class HealthResponse(BaseModel):
    status: str

database_is_reachable = True

@router.get("/health", response_model=HealthResponse)
def health() -> HealthResponse:
    return HealthResponse(status="ok")

@router.get("/ready", response_model=HealthResponse)
def ready(response: Response) -> HealthResponse:
    if not database_is_reachable:
        response.status_code = 503
        return HealthResponse(status="not ready")
    return HealthResponse(status="ok")
```

```
# main.py
from fastapi import FastAPI
from routes import router

app = FastAPI()
app.include_router(router)
```

```
# test_main.py
from fastapi.testclient import TestClient
from main import app
import routes
```

```
client = TestClient(app)

def test_health_always_ok():
    assert client.get("/health").status_code == 200

def test_ready_reflects_database_state():
    routes.database_is_reachable = False
    response = client.get("/ready")
    assert response.status_code == 503
    routes.database_is_reachable = True # reset for other tests
```

Expected output (`pytest test_main.py`):

2 passed

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate returns one honest, typed "yes I'm up" — correct, but it can never report a real problem. Approach 1 adds the liveness/readiness split, proving it with a fake-but-real dependency flag that actually changes the response. Approach 2 doesn't add new behavior — it reorganizes Approach 1's two routes into a separate `routes.py` via `APIRouter`, the exact file layout the Build Task's `Suggested files` section uses, and adds automated tests for both states instead of manual `curl` s.

Which one should you actually write? For this exercise alone, Intermediate Approach 1 is enough — it's what almost every real `/health` route looks like on day one. Reach for Advanced Approach 1's `/ready` split the moment your service actually depends on something that can go down without the process itself dying (a database, in this document's case). Reach for Approach 2's `APIRouter` layout as soon as you're about to add a second or third route — which happens immediately, in the very next exercise.